

EduTrack Onboarding — Bug Fix Report

2026-09-21 · Reported by @Someone

Overview

This document logs bugs found in the **Client Onboarding** module (`frontend/src/features/onboarding`), reported by the user during manual testing on the non-prod environment (`edutrack-nonprod.edubac.com`). Each entry below records the reported behaviour, the confirmed root cause in code, and a proposed fix or a question where the reported behaviour turns out to be intentional design.

Bug index

#	Screen	Problem
1	Project page — module list	The second module opens by default instead of the first
2	Dashboard — Ongoing projects panel	Delayed projects sit at the top; they should sit at the bottom
3	Project page — header	Status still reads "Not started" after the project has started
4	Projects list page — Status column	Status never moves off "Not started"
5	Dashboard — Your verifications	The numbers on the tile do not update
6	Task checklist — Required documents	The dialog does not say which document is required
7	Project page — Go-live sign-off	The sign-off panel is drawn twice
8	Task — Start task	A raw internal error naming "journey 12" is shown to the user

Bug 1 — Wrong module auto-expands when opening a project

Area: Project workspace — module accordion (`ObProjectWorkspace.tsx`)

Reported by: User, while opening a client project from **My Tasks / Projects**.

Description

When a project (e.g. `DPS-101`, screenshot below) is opened, the **second** module in the list ("Payment Gateway") is expanded automatically instead of the **first** module ("Student Information System"). The user expects the first module in the list to always open first.

Steps to reproduce

1. Open a project that has more than one module, where the first module is fully complete and a later module still has pending/partial work.
2. Navigate to it via **My Tasks** or **Projects** → client → project.
3. Observe which module accordion is open by default.

Expected vs actual

	Behaviour
Expected (per user)	The first module in the list is always the one that opens by default.
Actual	"Payment Gateway" (module 2) opens; "Student Information System" (module 1) stays collapsed, even though it is listed first.

Root cause

The page does not open module #1. It opens the first module that still has unfinished work. `firstServiceWithWork()` in `taskFilter.ts:94` walks the modules in list order and returns the first one holding a task that is not `DONE` or `SKIPPED`; the seeding effect in `ObProjectWorkspace.tsx:176-187` then opens that module once, on load.

In the screenshot, module 1 (Student Information System) was already 100% complete, so it was skipped and module 2 (Payment Gateway) opened instead. The behaviour was written on purpose — the component's own notes say the page should open on "the first module holding outstanding work" — but it is not what the team wants.

Decision

Confirmed by the reporter: **the first module in the list must always open first**, whatever its completion state. The two options considered were:

1. **Keep current design** — always open the first module *with pending/partial work* (skips finished modules). This matches the existing product intent documented in code.
2. **Change to literal first module** — always open module #1 in list order regardless of its completion state, per your report.

Fix: in the seeding effect of `ObProjectWorkspace.tsx` (lines 176–187), open `tree[0]` — the first module in the list — instead of the module returned by `firstServiceWithWork()`. The `?task=` deep link must keep its precedence, and nothing else on the page changes. The file belongs to Stream C (`tickets`), so the change goes through that stream.

Bug 2 — Delayed projects sit at the top of the Ongoing projects panel

Area: Dashboard → "ONGOING PROJECTS" tile → side panel

What happens now

Clicking the ONGOING PROJECTS tile opens the side panel with 7 projects. The late ones are listed first — "Somerville Project — 9 days late", then "DPS N Project — 5 days late" — and the "Ahead" and "On time" projects follow below them.

What should happen

Recent and running projects should be at the top of the list, and delayed projects should move to the bottom.

Root cause

`ObProjectListPanel.tsx:62` sorts the rows with `byLatenessDescending` (defined in `obProjectBoard.ts:238–252`), which is deliberately "worst first". The backend sends the rows unordered — `ObProjectBoardRepository.java` has no `ORDER BY` — so this single line decides the whole order.

Fix

Use a comparator in `ObProjectListPanel.tsx:62` that puts late projects last.

Careful: do not edit `byLatenessDescending` itself. `ObProjectSummaryLists.tsx:35-36` calls the same function, so changing it would also re-order those lists. Add a separate comparator for this panel only.

Bug 3 — Project header still reads "Not started" after the project has started

Area: Project page header — `ObProjectDetailPage.tsx:354`, chip at lines 386–393

What happens now

"Vishwa Bharti project" carries the chip **Not started** next to its title, although the project started on 20 Sep, both modules read **Complete**, and the same header shows 100% completion, 2/2 modules done and 5 of 5 tasks completed.

What should happen

Once work has begun, the chip should show the real state — running, on time, delayed or completed. "Not started" should only appear before any work starts.

Root cause

This is two defects stacked on each other.

1. **The chip does not read the project's status.** It reads the delay cell, `delayCell()` in `projectRow.ts:25-51`. Its very first branch (lines 26–32) returns the label "Not started" whenever `gateStatus === 'LOCKED'`, and returns before the `COMPLETED` branch at line 40 is ever reached. A locked gate therefore hides every other state.
2. **The gate is never opened by ordinary work.** `gateStatus` is not stored on the project; it is derived on every read in `ObProjectReadRepository.java:164-169` as `IF(SUM(gj.gate_status='OPEN')>0, 'OPEN', 'LOCKED')`. The only code that writes `gate_status='OPEN'` is `ObPrerequisiteGateService.java:113-114`, which runs when the client prerequisite checklist clears. But `ObJourneyStepLifecycleService.settleJourney` (lines 1990–2012) activates steps "whether or not its client's checklist has cleared". So tasks can run all the way to 100% while the gate stays `LOCKED`.

There is a third, related gap: `ObProject.complete()` in `backend/domain/.../onboarding/ObProject.java:241-245` is documented as being stamped when the last journey completes, but it has no callers in the backend. `settleJourney` stamps the journey's own completion and never touches the project, so `ob_projects.status` never becomes `COMPLETED` either.

Fix

Both sides need a change:

- **Backend** — make the project state follow the real work: open the gate when the first task starts (or stop deriving this label from the prerequisite gate), and call `ObProject.complete()` when the last journey completes.
- **Frontend** — in `delayCell()` (`projectRow.ts:25-32`), test the project's own status before the `LOCKED` branch, so a started or completed project can never print "Not started".

Bug 4 — Status column on the Projects list never updates

Area: Projects list page — `ObProjectListPage.tsx:294`, cell at lines 345-351

What happens now

The Status column shows "Not started" for projects that have already started, and it never changes as work progresses.

What should happen

The column should follow the project: not started → running / on time / delayed → completed.

Root cause

Same defect as **Bug 3**. The list column calls the same `delayCell()` in `projectRow.ts:25-51`, so the locked prerequisite gate returns "Not started" here too, before any other state is considered.

Fix

Fixing Bug 3 fixes this column as well — both screens read one function. Test both screens together: the project header chip and the list column must agree.

Bug 5 — "Your verifications" tile does not update

Area: Dashboard → "Your verifications" card — `ObReviewCards.tsx:39`

What happens now

The card shows "2 out for review", "6 approved today", "0 came back today". The figures do not move after the user approves or returns a check-list row.

What should happen

The figures should follow the user's own verification activity without a manual page reload.

Root cause

Two problems, both on the client side.

1. **The card never refetches.** `ObReviewCards.tsx:39` calls `useGetObReviewSummary()` with no query options, so it skips the shared dashboard settings in `obDashboardFreshness.ts:106-110` (`refetchOnMount: 'always'` plus a 60-second refetch) that every other card on the board passes. It falls back to the app default — 30-second stale time and no refetch on window focus.
2. **No invalidation after a verification.** `invalidateAfterTaskWrite` in `taskQueries.ts:45-55`, which every check-list writer calls, refreshes only the journey and My Tasks keys. It never calls `invalidateObDashboard`, so the card's cached answer survives the action that should change it.

Worth knowing for testing: the figures come from the pre-aggregated table

`ob_implementor_daily_stats` (GET `/onboarding/dashboard/review-summary` → `ObReviewSummaryService.java:44-54` → `ObReviewSummaryRepository.java:43-52`), which `ObStatsRefreshWorker.java:137` refreshes on roughly a 5-minute cycle. Live `COUNT(*)` on dashboards is not allowed in this project, so the number will always trail the action by up to one worker pass.

Fix

- Pass the shared `OB_DASHBOARD_QUERY` options to `useGetObReviewSummary()` in `ObReviewCards.tsx:39`, so the card polls like the rest of the board.
- Add `invalidateObDashboard` to `invalidateAfterTaskWrite` (`taskQueries.ts:45-55`) so a verification also refreshes this card.
- Expect the figure to move within the worker's refresh window, not instantly. If instant is required, that is a larger change to how the review counters are stored.

Bug 6 — Required documents dialog does not say which document is needed

Area: Task → More → Attach → "Required documents" dialog — `ObTaskActionDialogs.tsx:326`

What happens now

The dialog reads "Login Configuration — 0 of 1 satisfied". The single row is labelled only **Document**, with the status **Missing**, and the upload field underneath is also labelled **Document**. The user cannot tell which document is being asked for — Aadhaar card, PAN card, a signed agreement, or something else.

What should happen

Each row should name the document required, and the upload field should say which requirement the file will satisfy.

Root cause

Two separate things, and only one of them is a code defect.

1. **The row label is data, not a bug.** Line 376 prints `{doc.label}` — whatever label was stored on the template's document row. That field exists end to end: `ob_journey_template_step_docs.label` (migration `V20260903_1420__ob_journey_templates.sql:287-292`) → entity `ObJourneyTemplateStepDoc` → read SQL `ObJourneyReadRepository.java:331-357` → DTO `ObJourneyReadService.java:197-219` → TS model `obJourneyStepDoc.ts`. A row reading "Document" means the template was authored with the literal label "Document". **No schema or API change is needed for this half.**

2. **The upload field's label is hardcoded.** `ObTaskActionDialogs.tsx:391-392` prints the fixed word "Document" for the file input, whatever the requirement is called.

There is also a real limitation behind this, worth knowing before promising more: an uploaded file is not tied to a specific requirement. `isSatisfied` is counted, not matched — see the note at `ObJourneyReadRepository.java:320-326`. With two required documents, one upload satisfies "1 of 2" and nothing records which one.

Fix

- **Data (fixes what the user saw):** re-author the template's document rows with real names — "Aadhaar card", "PAN card", "Signed agreement" — in the journey template designer (`JourneyTemplateDesignerPage.tsx:2178-2188`) or through the Excel import, which already rejects blank labels (`ObJourneyTaskImportService.java:202-225`).
- **Code (small):** replace the hardcoded "Document" at `ObTaskActionDialogs.tsx:391-392` with the name of the requirement being uploaded.
- **Larger, separate:** matching an upload to one specific required document needs a new link table. Raise it as its own item if the team wants it; it is not part of this fix.

Bug 7 — "Go-live sign-off" panel appears twice

Area: Project page, below the modules — `ObProjectDetailPage.tsx:304-320` , panel in `SignoffPanel.tsx`

What happens now

"Vishwa Bharti project" shows two Go-live sign-off panels, one under the other. The first reads **Awaiting the client** (sent to Vishwa, link expires 4 Oct 2026) and the second reads **Not requested**, with a "Choose a contact..." dropdown and a "Request sign-off" button.

What should happen (as reported)

Only one Go-live sign-off panel on the project page.

Root cause

The page is not rendering the same panel twice. `ObProjectDetailPage.tsx:305-313` draws **one sign-off panel per completed module service**, and this project has two completed services — Student Information System and Payment Gateway. One service's

sign-off has been requested, the other has not, which is why the two panels show different states.

They look like duplicates because `SignoffPanel1.tsx:162` heads every panel with the same constant text, "Go-live sign-off", and never names the service it belongs to.

Decision needed

In the current workflow each module service carries its own sign-off record, so two completed services legitimately mean two sign-offs. There are two ways forward:

1. **Keep the workflow, fix the heading (recommended).** Name the module service in each panel — "Go-live sign-off — Student Information System". Nothing about the process changes; the reader can immediately see these are two different sign-offs. One-line frontend change.
2. **One sign-off for the whole project.** A single panel, sent once every module service is complete. This changes the workflow and the data model, so it needs product agreement before any code is written.

Because "do not change any workflow" applies to this round, **option 1 is what this document proposes.** If a single project-level sign-off is genuinely wanted, say so and it will be written up as its own change.

Bug 8 — Internal error text shown to the user on Start task

Area: Task dialog → Start task — message rendered at `ObTaskActionBar.tsx:279-283`

What happens now

Pressing **Start task** can print this under the buttons:

journey 12 is not open for step activity (held by journey 11)

It exposes internal database IDs and the word "journey", which is not a word the product uses on screen. Users have no way to know what journey 11 or 12 is.

What should happen

A sentence in the product's own vocabulary: which task cannot start, and which module service is holding it. No IDs.

Root cause

The backend's raw exception text is passed through to the screen untouched:

- The message is built in `JourneyNotOpenException.java:22-25` and thrown by `ObJourneyStepLifecycleService.java:257-259` on start.
- The handler `ObJourneyStepLifecycleExceptionHandler.java:137-144` returns 422 and copies the raw text into `problem.detail` (line 142).
- The frontend prints it verbatim: `taskActions.ts:435` returns `failed.problem.detail`, shown at `ObTaskActionBar.tsx:279-283` and as a toast at `StepActionBar.tsx:106`.

The names needed for a readable message are already in hand where the exception is thrown: `journey.getServiceName()` (the module service) and `step.getName()` (the task). Only the blocking journey is known by id alone (`heldByJourneyId`), so naming it needs one extra lookup of that journey's service name before the exception is built.

Fix

Build the message from names instead of ids, for example:

“Payment Gateway Setup” cannot start yet — it is waiting for the “Student Information System” module service to finish.

Keep the machine-readable `type: journey-not-open` on the problem response so the UI can still branch on it; only the human sentence changes. The MSW mock at `onboardingJourneyInstances.ts:105-106` mirrors the old wording and must be updated with it, or the frontend tests will assert text that no longer exists.